

## Project 2\_Lab L03-B01\_Muhammad Faris Syahmi Bin Norazizi\_52215125814,

**Date:** 9/6/2026

**Name :** Muhammad Faris Syahmi Bin Norazizi

**Repository Link:** <https://github.com/Syahmi813/Mini-Project-Mobile>

### Executive Summary

The **Mobile Catering Checkout System** is a cross-platform mobile application developed using the Flutter SDK. The system provides a self-service utility for clients to authenticate, browse a catalog of five curated culinary packages, configure custom event parameters via a sliding modal interface, calculate real-time dynamic pricing based on headcount, and process payments securely. Operating with an offline-first relational database layout, the app eliminates external network dependence while providing seamless screen transitions and immediate data synchronizations.

### 1. Project Overview & UI Walkthrough

The application's structural design and end-to-end user navigation flow follow a chronological, user-friendly layout architecture:

Plaintext

 VIEW FLOW LOGIC:

[Login Screen] —> [Home Dashboard] —> [Booking Modal] —> [Payment View] —> [Receipt View] —> [History Log]

#### 1.1 Secure Authentication Interface

The system handles security entries via a dedicated login view. The UI relies on crisp text fields with localized focus controls capturing a valid client email input (test@restaurant.com) alongside a masked character password sequence to ensure user data integrity.

#### 1.2 Five-Tier Menu Package Catalog

Upon successful verification, the app routes users directly to the main menu view. This screen dynamically greets the authenticated user ("Welcome, Alex Smith") and organizes five standalone culinary packages inside individual elevation cards:

- **Gold Jubilee Package:** \$150.0 per guest (5-Course Fine Dining, Premium Seafood Selection, and Custom Mocktails).
- **Diamond Banquet Package:** \$250.0 per guest (7-Course Luxury Caviar & Steak Tasting Menu, Custom Dessert Station).

- **Asian Fusion Feast:** \$120.0 per guest (Dim Sum Platters, Wagyu Beef Noodles, Peking Duck, and Matcha Gelato).
- **Mediterranean Gala:** \$180.0 per guest (Slow-roasted Lamb, Mezze Platters, Grilled Octopus, and Baklava Tower).
- **Plant-Based Oasis:** \$95.0 per guest (Organic Harvest Salads, Truffle Mushroom Risotto, and Artisanal Vegan Tarts).

### 1.3 Contextual Scheduling Engine

When a user interacts with a package option card, a contextual modal sheet rolls up smoothly from the base of the screen. This modal isolate configuration variables, wrapping user constraints into three clean metrics:

1. **Number of Guests** (via standard numeric text input).
2. **Event Date** (formatted as YYYY-MM-DD).
3. **Event Time** (formatted as HH:MM military time).

### 1.4 Real-Time Pricing & Masked Payment Processing

Once inputs are submitted, the app transitions to the payment interface. The application dynamically calculates the invoice amount using a standard dynamic multiplication formula:

$$\\$\\text{Amount Due} = \\text{Guest Count} \\times \\text{Base Price Per Guest}\\$\\$$$

For an order of 10 guests under the Gold Jubilee Package ( $10 \\times \$150.00$ ), the screen outputs an explicit **Amount Due: \$1500.00**. This view uses secure text grouping layout rules to automatically mask credit card data structures inside standard 4-digit token breaks (4111 2222 3333 4444) as the user types.

### 1.5 Itemized Success Receipts

Clicking the green payment confirmation button pushes the data directly to the local storage layer and redirects to the **Transaction Receipt** view. This screen features an isolated white container card highlighting an automated tracking ID (res\_17181002284), confirmation arrays, a total attendance indicator, and a prominent action button returning the user safely to the home dashboard.

### 1.6 Persistent History Logs & Data Wipes

Users can audit their finalized configurations at any time by selecting the "My Bookings" app bar button.

- If no configurations are recorded in the database, the workspace surfaces a clean state display featuring a stylized calendar icon and a "No active bookings found" disclaimer.
- Once checkouts are completed, the interface updates into a relational logging tree. Each logged block shows metadata chips detailing date, time, and headcount variables alongside a red **"Cancel Booking"** button. This component enables instant database row deletion and layout clearouts.

## 2. Relational Database Schema Design

The application's background operations run on a structured local SQLite architecture mapping out explicit data column.

### 2.1 Table: packages

| Column      | Data Type | Constraints | Description                           |
|-------------|-----------|-------------|---------------------------------------|
| id          | TEXT      | PRIMARY KEY | Unique menu identification token      |
| name        | TEXT      | NOT NULL    | Title of the specific package tier    |
| description | TEXT      | NOT NULL    | Breakdown of culinary dish inclusions |
| price       | REAL      | NOT NULL    | Dynamic base cost charged per guest   |

### 2.2 Table: bookings

| Column      | Data Type | Constraints | Description                                    |
|-------------|-----------|-------------|------------------------------------------------|
| id          | TEXT      | PRIMARY KEY | Unique generated transaction string            |
| packageName | TEXT      | NOT NULL    | Target reference matching the catering package |

| Column     | Data Type | Constraints | Description                                 |
|------------|-----------|-------------|---------------------------------------------|
| guestCount | INTEGER   | NOT NULL    | Total attendee headcount chosen by the user |
| eventDate  | TEXT      | NOT NULL    | Calendar date tracking string               |
| eventTime  | TEXT      | NOT NULL    | Clock timing execution string               |
| totalPaid  | REAL      | NOT NULL    | Calculated transaction invoice total        |

### 3. Core Technical Features

- **State Synchronization:** The user interface relies on declarative layout lifecycles that automatically track change states. This ensures that actions like clicking "Cancel Booking" wipe the underlying SQL row from memory and immediately re-render the view without requiring a manual pull-to-refresh or app restart.
- **Input Validation & Formatting:** Form text inputs are paired with native validators. This enforces chronological date constraints and controls the credit card text inputs to ensure only clean string structures are committed to storage.
- **Math Computation Stability:** Pricing calculations utilize double-precision floating-point numbers to prevent rounding errors or decimal truncation when multiplying guest variables against package rates.

## Conclusion

The development of the Mobile Catering Checkout System successfully demonstrates the viability of building a lightweight, high-performance, offline-first application by combining the Flutter SDK with a local SQLite relational database. By tightly coupling dynamic, declarative state lifecycles with local data transactions, the system achieves a highly responsive user experience with zero external network latency. Technical bottlenecks such as asynchronous initialization race conditions were completely resolved by enforcing sequential native loading sequences, while double-precision floating-point calculations ensured absolute mathematical precision for dynamic pricing summaries. Ultimately, this implementation provides a robust, production-ready framework for managing transactional event data locally, and its clean architectural structure leaves it perfectly positioned for future scaling vectors, such as the integration of live cloud synchronization clusters and formal payment gateway APIs.